

HAIC 제출 코드 전체 해설

실행 전략 · 계산식 · 상수 · 상태 · 전체 소스

문서 기준

현재 제출 추천 패키지 `submission-fast-completion.zip` 안에 들어 있는 7개 파일을 직접 읽어 작성했다. 작업 폴더의 최신 후보 코드와 섞지 않았다. 코드의 동작 설명과 전체 원문을 제공하며, 연구 과정이나 실험 이력은 다루지 않는다.

실제로 선택된 실행 조합

Agent → FastCompletionCoordination(preview_row_repair)
→ FixedHighSpeedRecoveryV2(impact_clear)
→ FixedHighSpeedAgent(damping)
→ 내부 객체 VisionCorridorAgent

읽는 순서

앞부분은 실제 실행되는 경로를 설명한다. 다음에는 패키지에 포함되지만 선택되지 않은 모드와 보조 함수를 설명한다. 마지막 부록에는 모든 파일의 전체 소스를 줄 번호와 함께 수록한다. 줄 번호는 각 원본 파일의 줄 번호다. 긴 줄의 지면상 줄바꿈은 원본 줄을 추가한 것이 아니다.

2026년 9월 29일

속도 수치는 게임 내부 단위다. 페달 명령의 크기와 실제 가속도는 구별한다.

목차

1 패키지 구성과 문서 범위	3
2 진입점, 입력과 출력	3
2.1 Agent의 네 메서드	3
2.2 관측 형식과 검사	3
3 화면에서 도로 찾기	4
3.1 밝기와 탐색 범위	4
3.2 두 가지 보간 함수	4
4 장애물 검출과 회피 방향	4
4.1 밝은 연결 영역 찾기	4
4.2 회피 방향 상태의 모든 조건	5
5 현재 속도 추정	5
6 하위 도로 추종기: VisionCorridorAgent	5
6.1 생성 파라미터와 유효성 검사	5
6.2 기본 조향과 하위 목표 속도	6
6.3 하위 페달과 실제 효과	6
7 조향 보정과 고정 속도 계층	6
7.1 실제로 선택된 damping 모드	6
7.2 첫 10회와 고정 페달	6
8 급감속과 충돌 가능성 처리	7
9 최상위 조정: preview_row_repair	7
9.1 정상 제동을 충돌로 오인하지 않기	7
9.2 42행이 사라진 경우	7
9.3 최종 목표 속도와 페달	8
9.4 계산 예시	8
10 한 번의 act에서 실제로 실행되는 순서	8
11 상태 변수와 reset	9
12 진단 정보의 의미와 주의점	9

13 포함되어 있지만 선택되지 않은 모드	10
13.1 FixedHighSpeedAgent의 다른 조향 모드	10
13.2 memory_reacquire	10
13.3 FastCompletionCoordination의 나머지 모드	10
14 사용되지 않는 특징 벡터 함수도 포함됨	11
14.1 7차원 시각 특징	11
14.2 1차원 시간 특징	11
14.3 배치 처리와 export 목록	12
15 예외, 미구현 기능, 실제 전략의 경계	12
16 최종 요약	12
A 패키지 식별 정보	13
B agent.py 전체 소스	14
C haic_agent/__init__.py 전체 소스	15
D haic_agent/fixed_high_speed_runtime.py 전체 소스	16
E haic_agent/corridor_agent.py 전체 소스	18
F haic_agent/pixel_features.py 전체 소스	22
G haic_agent/fixed_high_speed_recovery_v2.py 전체 소스	26
H haic_agent/fast_completion_coordination.py 전체 소스	27

1 패키지 구성과 문서 범위

이 패키지는 NumPy를 사용하는 규칙 기반 제어기다. 신경망 가중치, 학습 모델 로딩, 맵 좌표 파일, 경로 최적화기, 외부 통신은 포함되어 있지 않다. 부록의 전체 소스는 ZIP의 텍스트를 그대로 삽입했다. 공백·주석·사용하지 않는 함수도 생략하지 않았다.

파일	역할
agent.py	대회 호출 인터페이스와 실제 모드 선택
haic_agent/__init__.py	패키지 설명 문자열
haic_agent/pixel_features.py	입력 검사, 도로·장애물·속도 추출과 보조 특징 함수
haic_agent/corridor_agent.py	중심선 추종, 회피 방향 상태, 하위 속도 제어와 진단
haic_agent/fixed_high_speed_runtime.py	조향 보정과 고정 60 목표의 페달 제어
haic_agent/fixed_high_speed_recovery_v2.py	급감속 감지와 회피 조향 해제
haic_agent/fast_completion_coordination.py	제동 오인 방지, 누락 도로 복원, 최종 목표 속도

물리 엔진이나 환경 래퍼는 ZIP에 포함되지 않는다. 따라서 그 내부 코드를 이 패키지의 기능으로 설명하지 않는다. 이 문서의 “모든 코드”는 이 제출 ZIP의 일곱 파일 전체를 의미한다.

2 진입점, 입력과 출력

2.1 Agent의 네 메서드

`__init__`은 `FastCompletionCoordination("preview_row_repair")` 객체를 하나 만든다. `reset(observation)`은 내부 객체의 `reset`으로 전달한다. `act(observation)`은 내부 객체의 최종 행동을 그대로 반환한다. `last_step_diagnostics()`는 내부 진단 사전의 복사본을 반환한다. Agent 자체의 추가 판단은 없다.

2.2 관측 형식과 검사

`current_frame`은 입력을 NumPy float32 배열로 변환한다. 변환 중 `TypeError` 또는 `ValueError`는 설명 문자열을 붙인 `ValueError`로 바꾼다. 허용 모양은 (4, 84, 84) 또는 (84, 84)이다. 네 장일 때 마지막 장을 현재 화면으로 선택한다. 현재 화면의 모든 값이 유한하고 [0, 1]에 있어야 한다. 다른 크기나 범위는 예외다.

여기서 직접 검사하는 값은 **선택된 현재 화면**이다. 네 장 전체를 검증하는 것은 아니다. 이전 화면은 이후 속도 평균과 조향 보정에 사용되므로, 모든 입력 프레임이 정상이라는 환경 계약이 중요하다. 선택된 최상위 실행 경로에는 이 예외를 잡아 행동을 대신 반환하는 처리가 없다.

출력은 `np.asarray([steer, gas, brake], dtype=np.float32)`인 길이 3 배열이다. 최종 조향은 보통 [-0.7, 0.7], 가속은 [0, 0.6], 제동은 [0, 0.15] 또는 [0, 0.28] 범위다. 이들은 조작 명령이며 차량 위치나 실제 가속도를 직접 지정하지 않는다.

3 화면에서 도로 찾기

3.1 밝기와 탐색 범위

도로 픽셀은 $0.24 \leq I(y, x) \leq 0.52$ 로 분류한다. 화면 가로 기준은 $x = 42$ 이다. y 는 아래로 갈수록 증가한다. 탐색 행은 가까운 곳부터 54, 50, 46, 42, 38, 34, 30이다.

처음 탐색 중심을 42로 놓고, 각 행에서 이전에 찾은 중심의 좌우 17픽셀 안에 있는 도로 픽셀만 선택한다. 최소 4개 픽셀이 있으면 그 x 좌표들의 평균을 그 행의 중심으로 저장하고, 다음 행의 탐색 중심도 이 평균으로 바꾼다. 4개 미만이면 그 행은 사전에 넣지 않고 이전 탐색 중심을 유지한다.

이는 연속된 도로 경계의 중점을 엄밀히 구하는 방식은 아니다. 탐색 범위에 포함된 도로 밝기 픽셀의 평균이므로, 가림·다른 도로 조각·밝기 오분류가 중심에 영향을 줄 수 있다.

3.2 두 가지 보간 함수

`center_at(row, centers)`는 알려진 행을 오름차순 정렬하고 `np.interp`로 중심을 선형 보간한다. 알려진 행이 없으면 42를 반환한다. 범위 밖에서는 가장 끝의 값으로 유지된다. 장애물 높이에서 도로 중심을 구할 때 사용한다.

`observed_center(centers, row)`는 요청한 행이 있으면 바로 반환한다. 없으면 행 거리의 절댓값으로 정렬한 가장 가까운 두 점 a, b 로 다음을 계산한다.

$$\hat{c}_r = \text{clip} \left(c_a + \frac{(r - a)(c_b - c_a)}{b - a}, 0, 83 \right).$$

두 점이 없으면 `None`이다. 두 점이 모두 같은 쪽에 있으면 외삽이 된다. 현재 선택 모드에서는 42행 누락을 보완하는 데 사용한다.

4 장애물 검출과 회피 방향

4.1 밝은 연결 영역 찾기

도로 중심이 3개 미만이면 장애물을 찾지 않는다. 밝기 $I \geq 0.54$ 인 픽셀 중 $22 \leq y < 62$ 만 조사한다. 배열 전체 크기의 방문 표시를 만들고, 위에서 아래·왼쪽에서 오른쪽 순서로 픽셀을 탐색한다. 아직 방문하지 않은 밝은 픽셀마다 스택을 사용한 8방향 연결 영역 탐색을 수행한다.

각 영역의 면적, 최소·최대 좌표, 좌표 합을 계산한다. 다음 조건을 모두 만족해야 후보가 된다.

- 면적: 4 이상 80 이하 픽셀
- 바운딩 박스 너비: 2 이상 9 이하 픽셀
- 바운딩 박스 높이: 2 이상 10 이하 픽셀
- 영역 중심 x 와 그 높이의 도로 중심 사이: 12픽셀 이하

영역 중심은 면적에 포함된 픽셀 좌표의 평균이다. 최종 선택은 중심 y 가 가장 큰 후보다. 반환값은 (y_o, x_o, c_o) 또는 `None`이다. 실제 거리·장애물 ID·종류를 출력하지 않는다. 가장 아래에 보이는 후보를 가장 가까운 것으로 취급한다.

4.2 회피 방향 상태의 모든 조건

장애물의 도로 상대 위치를 $d = x_o - c_o$ 라 하자. $d < 0$ 이면 회피 부호 $q = +1$, 그 외에는 $q = -1$ 을 후보로 삼는다. 실제 코드의 조향 부호로 양수 쪽 또는 음수 쪽에 회피 항을 더한다.

1. 저장된 회피 부호가 0이면 새 후보를 채택하고 조우 횟수를 1 늘린다.
2. 부호가 이미 있고 $y_o < 52$ 이면 방향을 다시 판단한다. $|d| > 1$ 이면 후보 부호를 바로 채택한다.
3. $|d| \leq 1$ 이고 이전 상대 위치가 있으면 이동량 Δd 를 본다. 기존 $q > 0$, $d > -3$, $\Delta d > 2$ 이면 $q = -1$ 로 바꾼다. 기존 $q < 0$, $d < 3$, $\Delta d < -2$ 이면 $q = +1$ 로 바꾼다.
4. $y_o \geq 52$ 에서는 이미 정한 부호를 재평가하지 않는다. 상대 위치 기록은 갱신한다.
5. 검출되면 누락 횟수를 0으로 초기화하고 검출 횟수를 늘린다.
6. 검출이 사라지면 첫 누락에는 부호를 유지한다. 두 번째 연속 누락에서 부호·이전 상대 위치를 지운다. 누락된 프레임에는 회피 조향 항 자체를 더하지 않는다.

장애물 긴급도와 회피 조향 항은

$$u = \text{clip}((y_o - 22)/18, 0, 1), \quad s_o = q \cdot 0.34 \cdot u$$

이다. $y_o \geq 40$ 이면 긴급도는 이미 1이다. 현재 회피 배율은 1.0이다. 이 구조는 장애물 ID를 연결하는 추적기가 아니며, 다음 프레임에서 선택된 다른 물체에도 같은 상태가 사용될 수 있다.

5 현재 속도 추정

$\text{SPEED_ROI}=(77, 83, 10, 13)$ 을 슬라이스로 사용하므로 실제 영역은 행 77–82, 열 10–12다. 밝기 합을 M 이라 할 때 한 프레임의 추정 속도는

$$v_{\text{frame}} = \text{clip}((M - 0.27)/0.085, 0, 80).$$

네 장의 관측에서는 마지막 두 장 각각을 디코딩한 뒤 평균한다. 단일 화면이면 그 화면의 값 하나를 사용한다. 이 평균은 흔들림을 줄일 수 있지만 최근 변화에 지연을 더한다.

80은 이 **속도 추정 함수의 클리핑 상한**이다. 게임 엔진의 최고속도와 같은 개념이 아니다. 이 제출 코드에는 상한을 넘는 HUD를 별도로 해석하는 함수가 없다.

6 하위 도로 추종기: VisionCorridorAgent

6.1 생성 파라미터와 유효성 검사

기본값과 이 제출 경로가 실제로 전달하는 값은 다르다.

항목	기본값	실제 값	허용 범위
cruise speed	62	55	유한값 36–80
curve penalty	2	3	유한값 0–4
max gas	0.12	0.5	유한값 (0, 1]
obstacle steer scale	1	1	유한값 0–2
far obstacle speed	47	43	유한값 20–80
near obstacle speed	40	36	유한값 20–80

범위를 벗어나면 ValueError가 발생한다. 클래스의 도로·속도 밝기 상수와 같은 이름의 모듈 상수가 있지만, 실제 인식 래퍼는 pixel_features 함수를 호출한다. 클래스 상수만 바뀌도 인식 함수의 상수가 함께 바뀌는 구조는 아니다.

6.2 기본 조향과 하위 목표 속도

없는 행은 42를 대신 사용한다. $m = c_{42}$, $n = c_{54}$ 라 하면

$$s_r = 0.022(m - 42) + 0.018(m - n).$$

도로 추종 항에 검출된 장애물 회피 항을 더하고 $[-0.7, 0.7]$ 로 자른다. 즉 $s_{base} = \text{clip}(s_r + s_o, -0.7, 0.7)$ 이다.

하위 road_sweep는 $r \in \{30, 34, 38, 42\}$ 에 대해 $|c_r - c_{r+24}|$ 의 최댓값을 계산한다. 없는 행은 42다. 탐색 행에 58, 62, 66은 없으므로 해당 쌍의 두 번째 값은 42다. 이는 상위의 전체 중심 최댓값-최솟값과 다른 함수다.

하위 목표는 $\text{clip}(55 - 3 \text{road_sweep}, 36, 55)$ 이다. 장애물 높이 $y_o < 44$ 면 43, 그 외에는 36으로 상한을 낮춘다. 장애물 첫 누락에도 43 상한을 둔다. 이전 목표가 있으면 새 목표에 $0.65T_{old} + 0.35T_{new}$ 평활화를 적용한다.

6.3 하위 페달과 실제 효과

하위 함수는 $v > T + 1$ 일 때 가속 0, 제동 $\text{clip}(0.012(v - T), 0.04, 0.28)$ 을 반환한다. 그렇지 않으면 $v < T - 8$ 에서 $\text{gas}=0.5$, $v < T - 3$ 에서 $\text{gas}=0.5 \cdot 2/3$, 나머지에서 $\text{gas}=0.5 \cdot 5/12$ 이며 $\text{brake}=0$ 이다.

하지만 이 하위 가속·제동은 현재 제출 경로에서 매번 상위 함수로 덮어써진다. 최종 가속이 최대 0.5라고 해석하면 틀린다. 하위 조향, 장애물 상태와 진단 카운터는 사용되지만, 하위 목표의 평활화·43/36 제한·페달 분기는 최종 페달을 직접 결정하지 않는다.

7 조향 보정과 고정 속도 계층

7.1 실제로 선택된 damping 모드

상위 FixedHighSpeedAgent는 하위 act를 호출하고 그 조향에 보정 c 를 더해 다시 자른다. 11번째 act부터 현재 30·42·54행이 모두 있고 이전 화면에도 42행이 있으면

$$c = 0.025(c_{42}^{\text{current}} - c_{42}^{\text{previous}}).$$

조건을 만족하지 못하면 $c = 0$ 이다. 모드 이름은 damping이지만 실제 계산은 화면 중심의 프레임 간 변화에 대한 가산 항이다. 차량의 실제 요레이트나 타이어 슬립을 측정하는 제어는 아니다.

중요한 순서는 $\text{clip}(\text{clip}(s_r + s_o, -0.7, 0.7) + c, -0.7, 0.7)$ 이다. 두 번 클리핑하므로 모든 항을 한번에 더하고 자른 결과와 다를 수 있다.

7.2 첫 10회와 고정 페달

steps는 reset에서 0이다. 보정 조건은 증가 전 steps가 10 이상이고, 증가 연산은 고정 계층 act의 끝에 있다. 따라서 첫 10번 호출에는 시간 보정이 없고, 11번째부터 적용된다. 기본 도로 조향과 장애물 회피는 첫 호출부터 동작한다.

고정 계층은 매번 다음 페달을 만든다.

$$g_{60} = \text{clip}(0.12 + 0.04(60 - v), 0, 0.6), \quad b_{60} = \text{clip}(0.02(v - 62), 0, 0.15).$$

추정 속도가 0이면 $gas=0.6$, $brake=0$ 이다. 현재 제출 코드에는 $gas=1$ 로 시작하는 별도 출발 모드가 없다.

8 급감속과 충돌 가능성 처리

선택된 회복 모드는 `impact_clear`다. 이전 최대 4개 추정 속도를 저장한다. 다음 조건을 모두 만족하면 충돌 가능성을 감지한다.

- 증가 후 `steps`가 10 초과
- 현재 `impact` 카운터가 0
- 속도 이력이 비어 있지 않음
- 현재 추정 속도 $v < 50$
- 이전 4개 이력의 최댓값에서 현재 속도를 뺀 값이 10 초과

감지되면 `impact` 카운터를 12로 설정한다. 도로 42~54행이 모두 있고 카운터가 양수이면 장애물 회피 항을 빼고 $\text{clip}(s_r + c, -0.7, 0.7)$ 로 조향을 다시 계산한다. 카운터는 해당 호출 끝에서 1 감소하므로, 감지한 호출을 포함해 최대 12회 조건부 회피 해제를 적용한다. 도로가 누락된 호출에도 카운터 시간은 소비된다.

이는 실제 충돌 플래그를 읽지 않는 속도 기반 추정이다. 이 계층 자체는 가속·제동을 바꾸지 않는다. 실제로 멈췄는지 판단한 뒤 후진하거나 차량을 재배치하는 기능도 없다.

9 최상위 조정: `preview_row_repair`

9.1 정상 제동을 충돌로 오인하지 않기

위의 충돌 감지가 이번 호출에서 새로 발생했고 이전 최대 4개 최종 `brake` 중 하나라도 0.01보다 크면 감지를 취소한다. 조향을 회복 계층 적용 전의 `damping_steering`로 되돌리고 `impact` 카운터를 0으로 만든다. 현재 호출의 `brake`는 나중에 계산되므로 이 판단에는 아직 들어가지 않는다.

이 취소는 기존 `impact` 상태 전체를 매번 재검토하는 규칙이 아니다. 새 `trigger`가 생긴 순간에만 작동한다. 정당한 제동과 충돌이 가까운 시간에 함께 일어나면 실제 충돌도 가려질 가능성이 있다.

9.2 42행이 사라진 경우

`steps > 10`, 42행 없음, 54행 있음, 알려진 중심 최소 2개일 때 `observed_center`로 42행을 복원한다. 이 복원값과 54행, 현재 검출된 장애물 회피 항, 시간 보정으로 조향을 한 번에 다시 계산하고 $[-0.7, 0.7]$ 로 자른다.

복원된 중심은 원래 `centers` 사전에 삽입되지 않는다. 따라서 바로 뒤의 속도 계산은 원래 검출된 점만 사용한다. 이 모드에서는 54행까지 사라지면 같은 복원 분기가 작동하지 않는다. 모든 도로를 잃었을 때 장기간 방향을 기억하는 기능은 다른 선택 모드에만 있다.

9.3 최종 목표 속도와 페달

첫 10번 호출은 앞 계층의 60 목표 페달을 유지한다. 이후에는 검출된 도로 중심의 범위

$$S = \begin{cases} \max c_y - \min c_y, & \text{중심이 2개 이상,} \\ 28, & \text{그 외} \end{cases}$$

로 $T = \max(38, 60 - 0.8S)$ 를 계산한다. 장애물이 현재 검출되면 $T \leftarrow \min(T, 44)$ 이다. 중심이 거의 없으면 $60 - 0.8 \cdot 28 = 37.6$ 에서 바닥 제한으로 38이 된다. 장애물은 44보다 더 낮은 도로 목표를 올려 주지는 않는다.

$T < 60$ 이면 최종 페달을

$$g = \text{clip}(0.12 + 0.04(T - v), 0, 0.6), \quad b = \text{clip}(0.02(v - T - 2), 0, 0.28)$$

로 덮어쓴다. $T = 60$ 이면 기존의 brake 최대 0.15를 유지한다. 목표가 60보다 아주 조금 낮아지는 순간에도 제동 상한이 0.28로 바뀐다는 불연속이 있다.

가속·제동을 각각 독립 계산하므로 $T+2 < v < T+3$ 에서 둘 다 양수가 될 수 있다. 목표와 속도가 같아도 $\text{gas}=0.12$ 다. 이는 목표 속도를 유지하려는 가산항이지 물리적으로 교정된 유지 토크는 아니다.

9.4 계산 예시

조건	목표	속도	gas	brake
직선, 충분히 느림	60	40	0.60	0
직선, 목표 도달	60	60	0.12	0
직선, 초과 속도	60	70	0	0.15
장애물 제한	44	60	0	0.28
작은 초과 속도	44	46.5	0.02	0.01
큰 도로 중심 범위	38	45	0	0.10

표는 코드 식을 대입한 값이다. 이 명령이 만드는 실제 가속도나 정지거리를 측정한 표는 아니다.

10 한 번의 act에서 실제로 실행되는 순서

1. 최상위가 호출 전 impact 카운터를 보관한다. 선택 모드의 최종 판단에는 이 값이 직접 사용되지 않는다.
2. 고정 계층이 현재 프레임을 검증하고 도로 중심을 찾는다.
3. 내부 base가 다시 프레임·도로를 읽고 장애물을 검출한다. 회피 부호와 자체 목표·진단 상태를 갱신하고 행동을 반환한다.
4. 고정 계층이 시간 보정 조향과 60 목표 페달을 적용하고 steps를 증가시킨다.
5. 회복 계층이 급감속을 감지하고 필요하면 회피 조향을 해제한다. 속도 이력과 impact 카운터를 갱신한다.
6. 최상위가 최근 제동에 의한 감지 취소를 적용한다.
7. 필요한 경우 42행을 복원하고 조향을 다시 계산한다.

8. 도로 범위와 현재 장애물로 최종 목표 속도를 정해 페달을 덮어쓴다.
9. 마지막 조향, 최대 4개 최종 제동 이력, 진단 사건을 갱신하고 행동을 반환한다.

도로 검출과 속도 디코딩 일부가 중복 호출된다. 이 코드에는 이를 한 번만 계산해 공유하는 별도의 캐시 계층이 없다.

11 상태 변수와 reset

reset은 전달된 화면으로 위치를 학습하지 않고 상태를 초기화한다. 하위 base는 observation을 버린다. 상속 체인의 reset이 차례로 실행되어 다음 상태를 지운다.

상태	초기값과 역할
<code>_obstacle_side, _obstacle_missing</code>	0, 0. 회피 방향과 연속 검출 누락 수
<code>_last_obstacle_side_offset</code>	None. 이전 장애물의 도로 상대 위치
<code>_target_speed</code>	None. 하위 목표 평활화 기록
<code>_last_obstacle</code>	None. 현재 검출값; 각 act에서도 먼저 None으로 지움
<code>_obstacle_encounters, _obstacle_detection_frames</code>	0. 조우·검출 집계
<code>_speed_samples, _speed_sum, _speed_max</code>	0. 하위 속도 추정 집계
<code>_gas_frames, _brake_frames</code>	0. 하위 페달이 양수였던 횟수
<code>_last_speed_estimate, _last_target_speed, _last_gas, _last_brake</code>	0. 하위 마지막 판단 기록
<code>steps, last</code>	0, 빈 사전. 호출 수와 최상위에서 반환할 진단
<code>remembered_road, latched_turn, recovery_left, recovery_armed</code>	0, 0, 0, True. 기억 복귀 모드용; 현재 선택 모드에서는 복귀 명령을 켜지 않음
<code>centered_streak</code>	0. 7개 중심이 있고 42·54행 모두 화면 중심에서 8 이내이면 증가. 3회면 회복 카운터 초기화·재무장
<code>impact_left, speed_history</code>	0, 빈 리스트. 급감속 후 회피 해제 기간과 최근 4개 속도
<code>last_command, brake_history</code>	0, 빈 리스트. 마지막 최종 조향과 최근 4개 최종 brake
<code>recovering, stable, visible_steer, lost</code>	False, 0, 0, 0. 다른 조정 모드용 상태
<code>_rows</code>	

생성자 안의 `self.reset()`은 파이썬의 동적 메서드 호출에 의해 최상위 reset까지 실행된다. 각 reset이 super를 호출하므로 하위 상태도 초기화된다.

12 진단 정보의 의미와 주의점

Agent에서 노출하는 진단은 `dict(self.last)`다. 하위 base의 진단 함수는 별도로 존재하며 이 반환값에 자동 합쳐지지 않는다.

- 고정 계층: target speed, pixel speed, inherited steer, correction, changed, available, road centers, final gas/brake를 기록한다.

- 회복 계층: recovery mode, damping steer, recovery changed, recovery steps remaining, centered streak, impact proxy trigger, impact steps remaining을 추가한다.
- 최상위: coordination mode, 최종 target speed, braking proxy veto, geometry repaired, completion changed, 최종 gas/brake를 덮어쓰거나 추가한다.

changed는 고정 계층 보정 전후 차이, recovery_changed는 회복 계층 조향 변경, completion_changed는 최상위와 회복 계층 반환 행동의 차이이다. 어느 것도 전체 제출 전략과 별도 대조군 사이의 차이를 뜻하지 않는다.

제동 오인으로 impact를 취소하면 실제 impact_left는 0이 되지만 이미 기록된 impact_steps_remaining은 다시 고쳐지지 않는다. braking_proxy_veto와 실제 상태를 함께 해석해야 한다. 하위 gas/brake 집계는 상위가 덮어쓰기 전 수치이므로 최종 페달 사용률이 아니다.

base의 diagnostics()는 장애물 조우/검출, 속도 평균/최대, gas/brake 프레임 수를 반환한다. base의 last_step_diagnostics()는 하위 target, speed, gas, brake와 장애물 좌표/도로 중심/회피 부호를 반환한다. 값이 없으면 일부 항목은 None이다.

13 포함되어 있지만 선택되지 않은 모드

13.1 FixedHighSpeedAgent의 다른 조향 모드

- control: 시간 또는 앞보기 보정 없이 하위 조향을 쓴다.
- preview: 보정 $0.018(c_{30} - c_{42})$ 를 더한다.
- curvature: 보정 $0.018(c_{30} - 2c_{42} + c_{54})$ 를 더한다.
- pursuit: 원하는 조향을 $0.65 [48(c_{30} - 42) / ((c_{30} - 42)^2 + 24^2)]$ 로 정하고, 기본 road 항을 뺀 값을 보정으로 더한다. 화면 좌표 식이며 미터 단위 카메라 교정 주장은 없다.

이 모드들도 보정은 첫 10회 이후, 30·42·54행이 있을 때만 작동한다. 잘못된 모드 문자열은 ValueError다. 현재 인스턴스는 damping으로 고정된다.

13.2 memory_reacquire

42행이 사라지고 회복이 재무장된 상태이며 기억한 도로 조향의 절댓값이 0.01보다 크면, 그 방향으로 크기 0.35–0.7의 조향을 고정한다. 카운터는 80이다. 7개 중심이 보이고 42·54행이 중심에서 8 이내인 상태가 3회 이어지면 종료하고 다시 무장한다. 현재 선택된 impact_clear 모드에서는 이 분기를 실행하지 않는다.

13.3 FastCompletionCoordination의 나머지 모드

모든 모드는 하위 impact_clear와 damping을 포함한다. 이후 조정만 다르다.

모드	추가 동작
control	최상위 추가 제어 없이 60 목표를 유지
preview_budget	도로 중심 범위 및 장애물로 목표 38–60, 장애물 최대 44
preview_action_aware	preview budget + 최근 제동에 의한 급감속 감지 취소
actuator_rate	이전 최종 명령에서 조향 변화량을 호출당 ± 0.24 로 제한
impact_traction	호출 전 impact가 있거나 이번에 trigger가 발생하면 gas를 최대 0.18로 제한

모드	추가 동작
recovery_budget	54행 중심 오차가 6 초과 또는 42행 누락이면 회복 상태. 목표 40. 7개 중심과 오차 4 미만이 3회 연속이면 해제
recovery_geometry_aware preview_row_repair	제동 오인 취소 + 양쪽 중심 복원/조향 기억 + recovery budget 실제 선택. preview action aware + 조건부 42행 복원

recovery geometry aware는 42 또는 54행이 없으면 두 행을 각각 보간·외삽한다. 둘 다 구해지면 도로향, 회피향, 이전 42행과의 변화 $0.025(m - pm)$ 을 합친다. impact 중이며 취소되지 않았다면 회피향을 0으로 한다. 재계산한 조향을 기억한다. 두 점을 구할 수 없으면 최대 12회 기억 조향을 유지한다. 두 행이 정상적으로 보이면 현재 조향을 기억하고 누락 수를 초기화한다. 회복 예산의 54행 누락 기본값은 0이어서 오차가 42가 된다.

이 모드 이름들이 파일에 존재한다는 사실은 현재 ZIP에서 모두 실행된다는 뜻이 아니다. 선택은 agent.py와 생성자 체인이 결정한다.

14 사용되지 않는 특징 벡터 함수도 포함됨

14.1 7차원 시각 특징

extract_visual_features는 다음 값을 float32 배열로 반환한다. 현재 Agent의 act는 이 함수를 호출하지 않는다.

1. 평균 HUD 속도 / 80
2. $(c_{54} - 42)/42$, $[-1, 1]$ 로 클리핑
3. $(c_{42} - 42)/42$, $[-1, 1]$ 로 클리핑
4. 하위 road sweep / 42, $[0, 1]$ 로 클리핑
5. 장애물 존재: 1 또는 0
6. 장애물 상대 위치 $(x_o - c_o)/12$, $[-1, 1]$ 로 클리핑
7. 긴급도 $\text{clip}((y_o - 22)/18, 0, 1)$

장애물이 없으면 마지막 세 값은 모두 0이다. FEATURE_NAMES는 이 순서의 문자열 이름을 보관하고 VISUAL_FEATURE_SIZE는 7이다.

14.2 1차원 시간 특징

extract_temporal_features는 (4, 84, 84) 모양을 요구한다. 현재 장애물이 없거나 현재 정규화 상대 위치의 절댓값이 0.2 이상이면 0이다. 이전 화면에서 장애물을 찾지 못해도 0이다. 이전 상대 위치를 /12 후 $[-1, 1]$ 로 자르고, 그 절댓값이 0.3 미만이면 0, 그 외에는 이전 상대 위치 하나를 반환한다. 이름은 previous_obstacle_lateral_offset이고 크기는 1이다. 이 함수도 현재 주행에서 사용되지 않는다.

14.3 배치 처리와 export 목록

두 batch 함수는 $(B, 4, 84, 84)$ 모양과 전체 값의 유한성 $[0, 1]$ 범위를 검사하고 각 관측의 결과를 `np.stack`한다. 빈 배치를 별도로 처리하지 않으므로 빈 입력에 대한 정상 반환을 보장하지 않는다. 시간 특징의 단일 함수는 모양 검사만 명시적으로 수행한다.

`pixel_features.__all__`은 특징 이름/크기와 인식·특징 추출 함수들을 공개 이름으로 열거한다. `corridor_agent.__all__`은 `VisionCorridorAgent` 하나다. 이 목록은 실행 순서나 활성 모드를 선택하지 않는다. 파일 첫 설명의 “learned policy”라는 문장은 보조 특징 모듈의 설명이며, 이 제출 ZIP에 학습 정책이 존재한다는 증거가 아니다.

15 예외, 미구현 기능, 실제 전략의 경계

- `base` 단독의 `_frame`은 `ValueError`를 잡아 `None`을 반환하고 `base act`는 영벡터를 돌려준다. 그러나 현재 상위 `act`가 먼저 `current_frame`을 호출하므로 최종 Agent 전체가 항상 영벡터로 복구한다고 말할 수 없다.
- 도로 검출 누락은 해당 행 생략, 기본 중심 42, 조건부 42행 복원으로 처리한다. 모든 상황을 위한 탐색 주행은 없다.
- 장애물이 없거나 검출 조건에서 탈락하면 `None`이다. 별도 신뢰도나 여러 장애물의 공동 경로 계획은 없다.
- out-in-out 경로 생성, 전체 맵 기억, 곡률의 물리 단위 추정, 제동거리 기반 속도 계획, 모델 예측 제어는 이 ZIP에 없다.
- 실제 차량 속도·충돌·좌표를 환경에서 직접 읽지 않는다. 입력은 화면이고 명령 이력 및 내부 상태만 기억한다.
- 후진·재시작·순간이동·완주 여부 판정은 이 Agent의 기능이 아니다. 완주와 차량 물리는 외부 환경이 처리한다.
- 이 패키지는 실행 중 학습하거나 가중치를 갱신하지 않는다. 같은 관측과 상태에서는 동일한 계산식을 따른다.

16 최종 요약

최종 행동은 단일 식 하나가 아니라 하위 도로 추종, 시간 보정, 급감속 회피 해제, 제동 오인 취소, 누락 도로 복원, 최종 속도 조정이 순서대로 만든 결과다. 하위에서 계산한 행동 중 일부가 덮어써지므로, 코드에 존재하는 숫자와 실제 최종 조작에 적용되는 숫자를 구별해야 한다.

현재 전략을 한 문장으로 표현하면 다음과 같다: **픽셀 도로 중심을 따라 조향하고 가장 가까워 보이는 장애물의 반대쪽으로 회피 향을 더하며, 보이는 도로 중심의 좌우 범위로 속도 목표를 낮추고, 급감속·인식 누락 때 조향을 복구하는 규칙 기반 제어기다.**

A 패키지 식별 정보

ZIP SHA-256:

7bc38a3bdeab7cfd437cfbf5c06f15a81cfa18fad5317748ab13a259b8ea835

ZIP 크기: 9928 bytes. 파일 수: 7.

파일	bytes	줄 수
agent.py	388	6
haic_agent/__init__.py	32	1
haic_agent/fixed_high_speed_runtime.py	3105	70
haic_agent/corridor_agent.py	10439	258
haic_agent/pixel_features.py	9729	247
haic_agent/fixed_high_speed_recovery_v2.py	2906	61
haic_agent/fast_completion_coordination.py	5285	98

B agent.py 전체 소스

SHA-256:

a58359bb8469c60cff05e78d2749ff5e85287fabfc1756e064016f13aa69e579

```
1 from haic_agent.fast_completion_coordination import FastCompletionCoordination
2 class Agent:
3     def __init__(self): self.driver=FastCompletionCoordination("preview_row_repair")
4     def reset(self,observation): self.driver.reset(observation)
5     def act(self,observation): return self.driver.act(observation)
6     def last_step_diagnostics(self): return self.driver.last_step_diagnostics()
```

C haic_agent/_init_.py 전체 소스

SHA-256:

a5e2b46bb2f58049e8c627b3f3bd75f6b80ddbd6e8e9270b1b6dca19edee01d8

```
1 """Pixel controller runtime."""
```

D haic_agent/fixed_high_speed_runtime.py 전체 소스

SHA-256:

ab465ff850fb489ff5facde49b8979a2e1e164885015703edf33b81c531e8e88

```

1  """Pixel-only fixed-speed steering mechanisms for a TRAIN feasibility study."""
2  import numpy as np
3
4  from haic_agent.corridor_agent import VisionCorridorAgent
5  from haic_agent.pixel_features import current_frame, road_centers, estimate_observation_speed
6
7
8  ARMS = ("control", "preview", "curvature", "damping", "pursuit")
9  TARGET_SPEED = 60.0
10 PREFIX_DECISIONS = 10
11
12
13 def fixed_pedals(speed):
14     return (float(np.clip(0.12 + 0.04 * (TARGET_SPEED - speed), 0, 0.6)),
15             float(np.clip(0.02 * (speed - TARGET_SPEED - 2), 0, 0.15)))
16
17
18 class FixedHighSpeedAgent:
19     def __init__(self, mechanism="control"):
20         if mechanism not in ARMS:
21             raise ValueError("unknown steering mechanism")
22         self.mechanism = mechanism
23         self.base = VisionCorridorAgent(cruise_speed=55, curve_speed_penalty=3, max_gas=0.5,
24                                         obstacle_far_speed=43, obstacle_near_speed=36)
25         self.reset()
26
27     def reset(self, observation=None):
28         self.base.reset(observation)
29         self.steps = 0
30         self.last = {}
31
32     def act(self, observation):
33         frame = current_frame(observation)
34         centers = road_centers(frame)
35         inherited = self.base.act(observation)
36         near, middle = centers.get(54, 42.0), centers.get(42, 42.0)
37         baseline_road = 0.022 * (middle - 42) + 0.018 * (middle - near)
38         correction = 0.0
39         available = all(row in centers for row in (30, 42, 54))
40         if self.steps >= PREFIX_DECISIONS and available:
41             far = centers[30]
42             if self.mechanism == "preview":
43                 correction = 0.018 * (far - middle)
44             elif self.mechanism == "curvature":
45                 correction = 0.018 * (far - 2 * middle + near)
46             elif self.mechanism == "damping":
47                 pixels = np.asarray(observation)
48                 previous = road_centers(pixels[-2]) if pixels.ndim == 3 else {}
49                 if 42 in previous:
50                     correction = 0.025 * (middle - previous[42])
51             else:
52                 available = False
53             elif self.mechanism == "pursuit":
54                 # Image-space bearing; no claim of metric camera calibration.
55                 desired = 0.65 * (2 * 24 * (far - 42) / ((far - 42) ** 2 + 24 ** 2))
56                 correction = desired - baseline_road
57         steer = float(np.clip(float(inherited[0]) + correction, -self.base.MAX_STEER,
58                               self.base.MAX_STEER))
59         speed = estimate_observation_speed(observation, frame)
60         gas, brake = fixed_pedals(speed)
61         self.steps += 1
62         self.last = dict(target_speed=TARGET_SPEED, pixel_speed=float(speed),
63                           inherited_steer=float(inherited[0]), correction=correction,
64                           changed=abs(steer - float(inherited[0])) > 1e-6,

```

```
65         available=available, road_centers=centers,  
66         final_gas=gas, final_brake=brake)  
67     return np.asarray([steer, gas, brake], dtype=np.float32)  
68  
69     def last_step_diagnostics(self):  
70         return dict(self.last)
```

E haic_agent/corridor_agent.py 전체 소스

SHA-256:

00667a41e6ed07c715784669745dcf4646307214474daf2e87b73ee48dc835cd

```

1  """Pixel-only racing controller using road geometry, speed HUD, and obstacle cues."""
2
3  from __future__ import annotations
4
5  import numpy as np
6
7  from haic_agent.pixel_features import (
8      center_at as pixel_center_at,
9      current_frame as pixel_current_frame,
10     estimate_observation_speed as pixel_estimate_observation_speed,
11     estimate_speed as pixel_estimate_speed,
12     nearest_bright_object as pixel_nearest_bright_object,
13     road_centers as pixel_road_centers,
14     road_sweep as pixel_road_sweep,
15 )
16
17
18 class VisionCorridorAgent:
19     """Follow the road corridor and regulate speed from rendered pixels only."""
20
21     MAX_STEER = 0.7
22     ROAD_LOW = 0.24
23     ROAD_HIGH = 0.52
24     OBSTACLE_LOW = 0.54
25     OBSTACLE_STEER = 0.34
26     MAX_TEACHER_GAS = 0.12
27     MAX_TEACHER_BRAKE = 0.28
28     SPEED_ROI = (77, 83, 10, 13)
29     SPEED_BASELINE = 0.27
30     SPEED_PER_UNIT = 0.085
31
32     def __init__(
33         self,
34         *,
35         cruise_speed: float = 62.0,
36         curve_speed_penalty: float = 2.0,
37         max_gas: float = MAX_TEACHER_GAS,
38         obstacle_steer_scale: float = 1.0,
39         obstacle_far_speed: float = 47.0,
40         obstacle_near_speed: float = 40.0,
41     ) -> None:
42         if not np.isfinite(cruise_speed) or not 36.0 <= cruise_speed <= 80.0:
43             raise ValueError("cruise_speed must be finite and between 36 and 80")
44         if not np.isfinite(curve_speed_penalty) or not 0.0 <= curve_speed_penalty <= 4.0:
45             raise ValueError("curve_speed_penalty must be finite and between 0 and 4")
46         if not np.isfinite(max_gas) or not 0.0 < max_gas <= 1.0:
47             raise ValueError("max_gas must be finite and in (0, 1]")
48         if not np.isfinite(obstacle_steer_scale) or not 0.0 <= obstacle_steer_scale <= 2.0:
49             raise ValueError("obstacle_steer_scale must be finite and between 0 and 2")
50         for name, value in (
51             ("obstacle_far_speed", obstacle_far_speed),
52             ("obstacle_near_speed", obstacle_near_speed),
53         ):
54             if not np.isfinite(value) or not 20.0 <= value <= 80.0:
55                 raise ValueError(f"{name} must be finite and between 20 and 80")
56         self.cruise_speed = float(cruise_speed)
57         self.curve_speed_penalty = float(curve_speed_penalty)
58         self.max_gas = float(max_gas)
59         self.obstacle_steer_scale = float(obstacle_steer_scale)
60         self.obstacle_far_speed = float(obstacle_far_speed)
61         self.obstacle_near_speed = float(obstacle_near_speed)
62         self._obstacle_side = 0.0
63         self._obstacle_missing = 0
64         self._last_obstacle_side_offset: float | None = None

```

```

65     self._target_speed: float | None = None
66     self._obstacle_encounters = 0
67     self._obstacle_detection_frames = 0
68     self._speed_samples = 0
69     self._speed_sum = 0.0
70     self._speed_max = 0.0
71     self._gas_frames = 0
72     self._brake_frames = 0
73     self._last_obstacle: tuple[float, float, float] | None = None
74     self._last_speed_estimate = 0.0
75     self._last_target_speed = 0.0
76     self._last_gas = 0.0
77     self._last_brake = 0.0
78
79     def reset(self, observation=None) -> None:
80         """Clear route-dependent state between episodes."""
81         del observation
82         self._obstacle_side = 0.0
83         self._obstacle_missing = 0
84         self._last_obstacle_side_offset = None
85         self._target_speed = None
86         self._obstacle_encounters = 0
87         self._obstacle_detection_frames = 0
88         self._speed_samples = 0
89         self._speed_sum = 0.0
90         self._speed_max = 0.0
91         self._gas_frames = 0
92         self._brake_frames = 0
93         self._last_obstacle = None
94         self._last_speed_estimate = 0.0
95         self._last_target_speed = 0.0
96         self._last_gas = 0.0
97         self._last_brake = 0.0
98
99     @staticmethod
100     def _frame(observation) -> np.ndarray | None:
101         try:
102             return pixel_current_frame(observation)
103         except ValueError:
104             return None
105
106     def _road_centers(self, frame: np.ndarray) -> dict[int, float]:
107         return pixel_road_centers(frame)
108
109     @staticmethod
110     def _center_at(row: float, centers: dict[int, float]) -> float:
111         return pixel_center_at(row, centers)
112
113     def _nearest_bright_object(
114         self, frame: np.ndarray, centers: dict[int, float]
115     ) -> tuple[float, float, float] | None:
116         """Find the nearest small bright component inside the tracked road band."""
117         return pixel_nearest_bright_object(frame, centers)
118
119     @classmethod
120     def _estimate_speed(cls, frame: np.ndarray) -> float:
121         """Decode speed from the lower HUD bar at its calibrated pixel scale."""
122         return pixel_estimate_speed(frame)
123
124     def _observation_speed(self, observation, current_frame: np.ndarray) -> float:
125         return pixel_estimate_observation_speed(observation, current_frame)
126
127     @staticmethod
128     def _road_sweep(centers: dict[int, float]) -> float:
129         """Estimate upcoming bend strength from the pixel centerline spread."""
130         return pixel_road_sweep(centers)
131
132     def _pedals(self, speed: float, target_speed: float) -> tuple[float, float]:
133         if speed > target_speed + 1.0:
134             brake = float(np.clip((speed - target_speed) * 0.012, 0.04, self.MAX_TEACHER_BRAKE))

```

```

135         return 0.0, brake
136     if speed < target_speed - 8.0:
137         return self.max_gas, 0.0
138     if speed < target_speed - 3.0:
139         return self.max_gas * (2.0 / 3.0), 0.0
140     return self.max_gas * (5.0 / 12.0), 0.0
141
142 def act(self, observation) -> np.ndarray:
143     frame = self._frame(observation)
144     if frame is None:
145         return np.zeros(3, dtype=np.float32)
146
147     centers = self._road_centers(frame)
148     self._last_obstacle = None
149     far = centers.get(42, 42.0)
150     near = centers.get(54, 42.0)
151     steering = 0.022 * (far - 42.0) + 0.018 * (far - near)
152     target_speed = float(
153         np.clip(
154             self.cruise_speed - self.curve_speed_penalty * self._road_sweep(centers),
155             36.0,
156             self.cruise_speed,
157         )
158     )
159
160     obstacle = self._nearest_bright_object(frame, centers)
161     if obstacle is not None:
162         self._last_obstacle = obstacle
163         obstacle_y, obstacle_x, road_center = obstacle
164         # The image side of an obstacle can shift with perspective and bends.
165         # Re-evaluate to the near field, ignoring sub-pixel side noise.
166         if self._obstacle_side == 0.0:
167             self._obstacle_encounters += 1
168             side_offset = obstacle_x - road_center
169             candidate_side = 1.0 if side_offset < 0.0 else -1.0
170             if self._obstacle_side == 0.0:
171                 self._obstacle_side = candidate_side
172             elif obstacle_y < 52.0:
173                 if abs(side_offset) > 1.0:
174                     self._obstacle_side = candidate_side
175                 elif self._last_obstacle_side_offset is not None:
176                     offset_motion = side_offset - self._last_obstacle_side_offset
177                     if self._obstacle_side > 0.0 and side_offset > -3.0 and offset_motion > 2.0:
178                         self._obstacle_side = -1.0
179                     elif self._obstacle_side < 0.0 and side_offset < 3.0 and offset_motion < -2.0:
180                         self._obstacle_side = 1.0
181             self._last_obstacle_side_offset = side_offset
182             self._obstacle_detection_frames += 1
183             self._obstacle_missing = 0
184             urgency = float(np.clip((obstacle_y - 22.0) / 18.0, 0.0, 1.0))
185             steering += (
186                 self._obstacle_side
187                 * self.OBSTACLE_STEER
188                 * self.obstacle_steer_scale
189                 * urgency
190             )
191             target_speed = min(
192                 target_speed,
193                 self.obstacle_far_speed
194                 if obstacle_y < 44.0
195                 else self.obstacle_near_speed,
196             )
197         elif self._obstacle_side != 0.0:
198             self._obstacle_missing += 1
199             if self._obstacle_missing <= 1:
200                 target_speed = min(target_speed, 43.0)
201         else:
202             self._obstacle_side = 0.0
203             self._obstacle_missing = 0
204             self._last_obstacle_side_offset = None

```

```

205
206     if self._target_speed is not None:
207         target_speed = 0.65 * self._target_speed + 0.35 * target_speed
208     self._target_speed = target_speed
209
210     speed = self._observation_speed(observation, frame)
211     self._last_speed_estimate = speed
212     self._speed_samples += 1
213     self._speed_sum += speed
214     self._speed_max = max(self._speed_max, speed)
215     gas, brake = self._pedals(speed, target_speed)
216     self._last_target_speed = target_speed
217     self._last_gas = gas
218     self._last_brake = brake
219     self._gas_frames += int(gas > 0.0)
220     self._brake_frames += int(brake > 0.0)
221     return np.asarray(
222         [
223             np.clip(steering, -self.MAX_STEER, self.MAX_STEER),
224             gas,
225             brake,
226         ],
227         dtype=np.float32,
228     )
229
230     def diagnostics(self) -> dict[str, float | int]:
231         """Expose pixel-controller counters for evaluation without simulator state."""
232         return {
233             "obstacle_encounters": self._obstacle_encounters,
234             "obstacle_detection_frames": self._obstacle_detection_frames,
235             "pixel_speed_mean": (
236                 self._speed_sum / self._speed_samples if self._speed_samples else 0.0
237             ),
238             "pixel_speed_max": self._speed_max,
239             "gas_frames": self._gas_frames,
240             "brake_frames": self._brake_frames,
241         }
242
243     def last_step_diagnostics(self) -> dict[str, float | None]:
244         """Expose the last pixel-based control decision for offline trace analysis."""
245         obstacle = self._last_obstacle
246         return {
247             "target_speed": self._last_target_speed,
248             "pixel_speed": self._last_speed_estimate,
249             "gas": self._last_gas,
250             "brake": self._last_brake,
251             "obstacle_y": obstacle[0] if obstacle is not None else None,
252             "obstacle_x": obstacle[1] if obstacle is not None else None,
253             "road_center_at_obstacle": obstacle[2] if obstacle is not None else None,
254             "obstacle_steer_side": self._obstacle_side or None,
255         }
256
257
258     __all__ = ["VisionCorridorAgent"]

```

F haic_agent/pixel_features.py 전체 소스

SHA-256:

f68895b9c0297ac5075ea47ab7f2a868e739b0ad24bee980082ce818b661d22a

```

1  """Deterministic perception features derived only from the 84x84 pixel view.
2
3  These values describe HUD and road objects. They never select or modify an
4  action; the learned policy remains responsible for driving decisions.
5  """
6
7  from __future__ import annotations
8
9  import numpy as np
10
11
12  ROAD_LOW = 0.24
13  ROAD_HIGH = 0.52
14  OBSTACLE_LOW = 0.54
15  SPEED_ROI = (77, 83, 10, 13)
16  SPEED_BASELINE = 0.27
17  SPEED_PER_UNIT = 0.085
18  FEATURE_NAMES = (
19      "speed_fraction",
20      "near_road_center_offset",
21      "far_road_center_offset",
22      "road_curve_magnitude",
23      "obstacle_present",
24      "obstacle_lateral_offset",
25      "obstacle_urgency",
26  )
27  VISUAL_FEATURE_SIZE = len(FEATURE_NAMES)
28  TEMPORAL_FEATURE_NAMES = ("previous_obstacle_lateral_offset",)
29  TEMPORAL_FEATURE_SIZE = len(TEMPORAL_FEATURE_NAMES)
30
31
32  def current_frame(observation: np.ndarray) -> np.ndarray:
33      """Validate an observation and return its newest normalized grayscale frame."""
34      try:
35          pixels = np.asarray(observation, dtype=np.float32)
36      except (TypeError, ValueError) as error:
37          raise ValueError("observation must contain normalized float pixels") from error
38      if pixels.shape == (4, 84, 84):
39          frame = pixels[-1]
40      elif pixels.shape == (84, 84):
41          frame = pixels
42      else:
43          raise ValueError("observation must have shape (84, 84) or (4, 84, 84)")
44      if not np.all(np.isfinite(frame)) or float(frame.min()) < 0.0 or float(frame.max()) > 1.0:
45          raise ValueError("observation pixels must be finite and normalized to [0, 1]")
46      return frame
47
48
49  def road_centers(frame: np.ndarray) -> dict[int, float]:
50      """Track the visible asphalt center from near to far image rows."""
51      asphalt = (frame >= ROAD_LOW) & (frame <= ROAD_HIGH)
52      horizontal = np.arange(frame.shape[1], dtype=np.float32)
53      centers: dict[int, float] = {}
54      previous = 42.0
55      for row in (54, 50, 46, 42, 38, 34, 30):
56          selected = asphalt[row] & (np.abs(horizontal - previous) <= 17.0)
57          locations = np.flatnonzero(selected)
58          if len(locations) >= 4:
59              previous = float(locations.mean())
60              centers[row] = previous
61      return centers
62
63
64  def center_at(row: float, centers: dict[int, float]) -> float:

```

```

65     """Interpolate a road center for a fractional image row."""
66     known_rows = sorted(centers)
67     if not known_rows:
68         return 42.0
69     return float(
70         np.interp(
71             row,
72             np.asarray(known_rows, dtype=np.float32),
73             np.asarray([centers[y] for y in known_rows], dtype=np.float32),
74         )
75     )
76
77
78 def nearest_bright_object(
79     frame: np.ndarray, centers: dict[int, float]
80 ) -> tuple[float, float, float] | None:
81     """Find the nearest small bright component inside the visible road corridor."""
82     if len(centers) < 3:
83         return None
84     bright = frame >= OBSTACLE_LOW
85     bright[:22, :] = False
86     bright[62:, :] = False
87     visited = np.zeros(bright.shape, dtype=np.bool_)
88     candidates: list[tuple[float, float, float]] = []
89     height, width = bright.shape
90
91     for start_y in range(22, 62):
92         for start_x in range(width):
93             if not bright[start_y, start_x] or visited[start_y, start_x]:
94                 continue
95             stack = [(start_x, start_y)]
96             visited[start_y, start_x] = True
97             min_x = max_x = start_x
98             min_y = max_y = start_y
99             sum_x = sum_y = area = 0
100             while stack:
101                 x, y = stack.pop()
102                 area += 1
103                 sum_x += x
104                 sum_y += y
105                 min_x = min(min_x, x)
106                 max_x = max(max_x, x)
107                 min_y = min(min_y, y)
108                 max_y = max(max_y, y)
109                 for neighbor_y in range(max(22, y - 1), min(62, y + 2)):
110                     for neighbor_x in range(max(0, x - 1), min(width, x + 2)):
111                         if bright[neighbor_y, neighbor_x] and not visited[neighbor_y, neighbor_x]:
112                             visited[neighbor_y, neighbor_x] = True
113                             stack.append((neighbor_x, neighbor_y))
114
115             box_width = max_x - min_x + 1
116             box_height = max_y - min_y + 1
117             if not (4 <= area <= 80 and 2 <= box_width <= 9 and 2 <= box_height <= 10):
118                 continue
119             center_x = sum_x / area
120             center_y = sum_y / area
121             road_center = center_at(center_y, centers)
122             if abs(center_x - road_center) <= 12.0:
123                 candidates.append((center_y, center_x, road_center))
124
125     return max(candidates, key=lambda item: item[0]) if candidates else None
126
127
128 def estimate_speed(frame: np.ndarray) -> float:
129     """Decode the rendered speed bar using its calibrated grayscale mass."""
130     top, bottom, left, right = SPEED_ROI
131     mass = float(np.asarray(frame, dtype=np.float32)[top:bottom, left:right].sum())
132     return float(np.clip((mass - SPEED_BASELINE) / SPEED_PER_UNIT, 0.0, 80.0))
133
134

```

```

135 def estimate_observation_speed(observation: np.ndarray, frame: np.ndarray) -> float:
136     """Average the last two rendered speed readings in the frame stack."""
137     pixels = np.asarray(observation, dtype=np.float32)
138     stack = pixels[-2:] if pixels.shape == (4, 84, 84) else pixels[np.newaxis, ...]
139     return float(np.mean([estimate_speed(item) for item in stack]))
140
141
142 def road_sweep(centers: dict[int, float]) -> float:
143     """Estimate bend strength from visible near/far road-center changes."""
144     return max(
145         (
146             abs(centers.get(row, 42.0) - centers.get(row + 24, 42.0))
147             for row in (30, 34, 38, 42)
148         ),
149         default=0.0,
150     )
151
152
153 def extract_visual_features(observation: np.ndarray) -> np.ndarray:
154     """Encode HUD speed, road alignment, bend, and nearest obstacle from pixels."""
155     frame = current_frame(observation)
156     centers = road_centers(frame)
157     obstacle = nearest_bright_object(frame, centers)
158     if obstacle is None:
159         obstacle_present = 0.0
160         obstacle_lateral = 0.0
161         obstacle_urgency = 0.0
162     else:
163         obstacle_y, obstacle_x, obstacle_road_center = obstacle
164         obstacle_present = 1.0
165         obstacle_lateral = float(
166             np.clip((obstacle_x - obstacle_road_center) / 12.0, -1.0, 1.0)
167         )
168         obstacle_urgency = float(np.clip((obstacle_y - 22.0) / 18.0, 0.0, 1.0))
169     return np.asarray(
170         (
171             estimate_observation_speed(observation, frame) / 80.0,
172             np.clip((centers.get(54, 42.0) - 42.0) / 42.0, -1.0, 1.0),
173             np.clip((centers.get(42, 42.0) - 42.0) / 42.0, -1.0, 1.0),
174             np.clip(road_sweep(centers) / 42.0, 0.0, 1.0),
175             obstacle_present,
176             obstacle_lateral,
177             obstacle_urgency,
178         ),
179         dtype=np.float32,
180     )
181
182
183 def extract_visual_features_batch(observations: np.ndarray) -> np.ndarray:
184     """Extract a row per observation for a batch of ``(B, 4, 84, 84)`` pixels."""
185     pixels = np.asarray(observations, dtype=np.float32)
186     if pixels.ndim != 4 or pixels.shape[1:] != (4, 84, 84):
187         raise ValueError("observations must have shape (B, 4, 84, 84)")
188     if not np.all(np.isfinite(pixels)) or float(pixels.min()) < 0.0 or float(pixels.max()) > 1.0:
189         raise ValueError("observation pixels must be finite and normalized to [0, 1]")
190     return np.stack([extract_visual_features(item) for item in pixels], axis=0)
191
192
193 def extract_temporal_features(observation: np.ndarray) -> np.ndarray:
194     """Read a prior-frame obstacle side only when the current view is centered."""
195     pixels = np.asarray(observation, dtype=np.float32)
196     if pixels.shape != (4, 84, 84):
197         raise ValueError("observation must have shape (4, 84, 84)")
198     current = pixels[-1]
199     previous = pixels[-2]
200     current_centers = road_centers(current)
201     current_obstacle = nearest_bright_object(current, current_centers)
202     if current_obstacle is None:
203         return np.zeros(TEMPORAL_FEATURE_SIZE, dtype=np.float32)
204     _, current_x, current_road_center = current_obstacle

```

```

205     current_lateral = float((current_x - current_road_center) / 12.0)
206     if abs(current_lateral) >= 0.2:
207         return np.zeros(TEMPORAL_FEATURE_SIZE, dtype=np.float32)
208     previous_centers = road_centers(previous)
209     previous_obstacle = nearest_bright_object(previous, previous_centers)
210     if previous_obstacle is None:
211         return np.zeros(TEMPORAL_FEATURE_SIZE, dtype=np.float32)
212     _, previous_x, previous_road_center = previous_obstacle
213     previous_lateral = float(
214         np.clip((previous_x - previous_road_center) / 12.0, -1.0, 1.0)
215     )
216     if abs(previous_lateral) < 0.3:
217         return np.zeros(TEMPORAL_FEATURE_SIZE, dtype=np.float32)
218     return np.asarray((previous_lateral,), dtype=np.float32)
219
220
221 def extract_temporal_features_batch(observations: np.ndarray) -> np.ndarray:
222     """Extract one temporal side feature per ``(B, 4, 84, 84)`` observation."""
223     pixels = np.asarray(observations, dtype=np.float32)
224     if pixels.ndim != 4 or pixels.shape[1:] != (4, 84, 84):
225         raise ValueError("observations must have shape (B, 4, 84, 84)")
226     if not np.all(np.isfinite(pixels)) or float(pixels.min()) < 0.0 or float(pixels.max()) > 1.0:
227         raise ValueError("observations pixels must be finite and normalized to [0, 1]")
228     return np.stack([extract_temporal_features(item) for item in pixels], axis=0)
229
230
231 __all__ = [
232     "FEATURE_NAMES",
233     "TEMPORAL_FEATURE_NAMES",
234     "TEMPORAL_FEATURE_SIZE",
235     "VISUAL_FEATURE_SIZE",
236     "center_at",
237     "current_frame",
238     "estimate_observation_speed",
239     "estimate_speed",
240     "extract_visual_features",
241     "extract_visual_features_batch",
242     "extract_temporal_features",
243     "extract_temporal_features_batch",
244     "nearest_bright_object",
245     "road_centers",
246     "road_sweep",
247 ]

```

G haic_agent/fixed_high_speed_recovery_v2.py 전체 소스

SHA-256:

dd1da6da44e009043625af401f5d1cf1cd09b915608b7fdbb42a2a8f2b46e0fc

```

1  """Stateful reacquisition and stale-avoidance clearing at the same high target."""
2  import numpy as np
3  from haic_agent.fixed_high_speed_runtime import FixedHighSpeedAgent
4
5  MODES = ("memory_reacquire", "impact_clear")
6
7
8  class FixedHighSpeedRecoveryV2(FixedHighSpeedAgent):
9      def __init__(self, mechanism="memory_reacquire"):
10         if mechanism not in MODES:
11             raise ValueError("unknown recovery mode")
12         self.recovery_mode = mechanism
13         super().__init__("damping")
14
15     def reset(self, observation=None):
16         super().reset(observation)
17         self.remembered_road = 0.0
18         self.latched_turn = 0.0
19         self.recovery_left = 0
20         self.recovery_armed = True
21         self.centered_streak = 0
22         self.impact_left = 0
23         self.speed_history = []
24
25     def act(self, observation):
26         action = super().act(observation)
27         original = float(action[0])
28         centers = self.last["road_centers"]
29         speed = self.last["pixel_speed"]
30         complete = 42 in centers and 54 in centers
31         centered = len(centers)==7 and abs(centers[42]-42)<=8 and abs(centers[54]-42)<=8
32         self.centered_streak = self.centered_streak+1 if centered else 0
33         if self.centered_streak >= 3:
34             self.recovery_left = 0
35             self.recovery_armed = True
36         trigger = False
37         if (self.steps>10 and self.impact_left==0 and self.speed_history and speed<50
38             and max(self.speed_history)-speed>10):
39             self.impact_left = 12
40             trigger = True
41         if self.steps>10 and self.recovery_mode=="memory_reacquire":
42             if (42 not in centers and self.recovery_armed and abs(self.remembered_road)>.01):
43                 self.recovery_left = 80
44                 self.recovery_armed = False
45                 self.latched_turn = np.sign(self.remembered_road)*np.clip(abs(self.remembered_road),.35,.7)
46             if self.recovery_left>0:
47                 action[0] = self.latched_turn
48                 self.recovery_left -= 1
49         elif self.steps>10 and self.recovery_mode=="impact_clear" and self.impact_left>0 and complete:
50             road = .022*(centers[42]-42)+.018*(centers[42]-centers[54])
51             action[0] = np.clip(road+self.last["correction"],-.7,.7)
52         if complete and self.recovery_left==0:
53             self.remembered_road = .022*(centers[42]-42)+.018*(centers[42]-centers[54])
54         if self.impact_left:
55             self.impact_left -= 1
56         self.speed_history = (self.speed_history+[speed])[-4:]
57         self.last.update(recovery_mode=self.recovery_mode,damping_steering=original,
58             recovery_changed=abs(float(action[0])-original)>1e-6,
59             recovery_steps_remaining=self.recovery_left,centered_streak=self.centered_streak,
60             impact_proxy_trigger=trigger,impact_steps_remaining=self.impact_left)
61         return action

```

H haic_agent/fast_completion_coordination.py 전체 소스

SHA-256:

2f890619535b744977ecbee04c1604f9bed8f2be94fd92de733dfb1c0c895d84

```

1  """Coordinate high-speed pedals and steering using only rendered road cues."""
2  import numpy as np
3  from haic_agent.fixed_high_speed_recovery_v2 import FixedHighSpeedRecoveryV2
4  from haic_agent.pixel_features import road_centers
5
6  MODES=('control','preview_budget','actuator_rate','impact_traction','recovery_budget',
7         'preview_action_aware','recovery_geometry_aware','preview_row_repair')
8
9
10 def observed_center(centers,row):
11     if row in centers: return centers[row]
12     known=sorted(centers,key=lambda y:abs(y-row))
13     if len(known)<2: return None
14     a,b=known[:2]
15     return float(np.clip(centers[a]+(row-a)*(centers[b]-centers[a])/(b-a),0,83))
16
17
18 class FastCompletionCoordination(FixedHighSpeedRecoveryV2):
19     def __init__(self,mechanism='control'):
20         if mechanism not in MODES: raise ValueError('unknown mechanism')
21         self.coordination_mode=mechanism
22         super().__init__('impact_clear')
23
24     def reset(self,observation=None):
25         super().reset(observation)
26         self.last_command=0.
27         self.recovering=False
28         self.stable=0
29         self.brake_history=[]
30         self.visible_steer=0.
31         self.lost_rows=0
32
33     def act(self,observation):
34         impact_before=self.impact_left
35         action=super().act(observation)
36         original=action.copy()
37         centers=self.last['road_centers']
38         speed=self.last['pixel_speed']
39         target=60.
40         veto=False
41         geometry_repaired=False
42         if self.steps>10:
43             if self.coordination_mode in ('preview_action_aware','recovery_geometry_aware','
44             preview_row_repair'):
45                 veto=self.last['impact_proxy_trigger'] and any(b>.01 for b in self.brake_history)
46                 if veto:
47                     action[0]=self.last['damping_steer']
48                     self.impact_left=0
49             if self.coordination_mode=='preview_row_repair' and 42 not in centers and 54 in centers and len
50             (centers)>=2:
51                 m=observed_center(centers,42)
52                 obstacle=self.base._last_obstacle
53                 term=self.base._obstacle_side*.34*float(np.clip((obstacle[0]-22)/18,0,1)) if obstacle is
54                 not None else 0.
55                 action[0]=np.clip(.022*(m-42)+.018*(m-centers[54])+term+self.last['correction'],-.7,.7)
56                 geometry_repaired=True
57             if self.coordination_mode=='recovery_geometry_aware':
58                 if 42 not in centers or 54 not in centers:
59                     n,m=observed_center(centers,54),observed_center(centers,42)
60                     if n is not None and m is not None:
61                         stack=np.asarray(observation)
62                         prev=road_centers(stack[-2]) if stack.ndim==3 else centers
63                         pm=observed_center(prev,42)
64                         obstacle=self.base._last_obstacle

```

```

62         clearing=(impact_before>0 or self.last['impact_proxy_trigger']) and not veto
63         term=self.base._obstacle_side*.34*float(np.clip((obstacle[0]-22)/18,0,1)) if
obstacle is not None and not clearing else 0.
64         action[0]=np.clip(.022*(m-42)+.018*(m-n)+term+(.025*(m-pm) if pm is not None else
0.),-.7,.7)
65         self.visible_steer=float(action[0]);self.lost_rows=0
66         geometry_repaired=True
67         else:
68             self.lost_rows+=1
69             if self.lost_rows<=12:
70                 action[0]=self.visible_steer
71                 geometry_repaired=True
72             else:
73                 self.visible_steer=float(action[0]);self.lost_rows=0
74         if self.coordination_mode in ('preview_budget','preview_action_aware','preview_row_repair'):
75             sweep=max(centers.values())-min(centers.values()) if len(centers)>1 else 28.
76             target=max(38.,60.-.8*sweep)
77             if self.base._last_obstacle is not None: target=min(target,44.)
78         elif self.coordination_mode=='actuator_rate':
79             action[0]=np.clip(action[0],self.last_command-.24,self.last_command+.24)
80         elif self.coordination_mode=='impact_traction':
81             if impact_before>0 or self.last['impact_proxy_trigger']:
82                 action[1]=min(action[1],.18)
83         elif self.coordination_mode in ('recovery_budget','recovery_geometry_aware'):
84             error=abs(centers.get(54,0)-42)
85             if error>6 or 42 not in centers: self.recovering=True
86             self.stable=self.stable+1 if len(centers)==7 and error<4 else 0
87             if self.stable>=3: self.recovering=False
88             if self.recovering: target=40.
89         if target<60:
90             action[1]=np.clip(.12+.04*(target-speed),0,.6)
91             action[2]=np.clip(.02*(speed-target-2),0,.28)
92         self.last_command=float(action[0])
93         self.brake_history=(self.brake_history+[float(action[2])])[-4:]
94         self.last.update(coordination_mode=self.coordination_mode,target_speed=target,
95                         braking_proxy_veto=bool(veto),geometry_repaired=geometry_repaired,
96                         completion_changed=bool(np.max(np.abs(action-original))>1e-6),
97                         final_gas=float(action[1]),final_brake=float(action[2]))
98         return action

```